

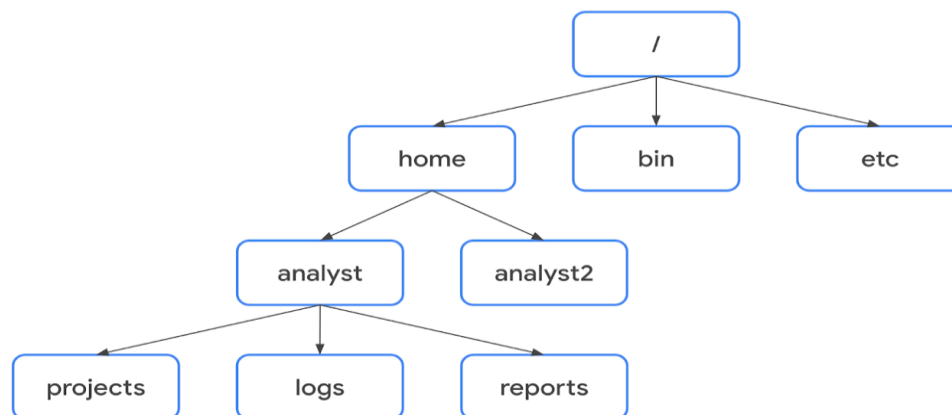
Navigate Linux and read file content

In this reading, you'll review how to navigate the file system using Linux commands in Bash. You'll further explore the organization of the Linux Filesystem Hierarchy Standard, review several common Linux commands for navigation and reading file content, and learn a couple of new commands.

Filesystem Hierarchy Standard (FHS)

Previously, you learned that the **Filesystem Hierarchy Standard (FHS)** is the component of Linux that organizes data. The FHS is important because it defines how directories, directory contents, and other storage is organized in the operating system.

This diagram illustrates the hierarchy of relationships under the FHS:



Under the FHS, a file's location can be described by a file path. A **file path** is the location of a file or directory. In the file path, the different levels of the hierarchy are separated by a forward slash (/).

Root directory

The **root directory** is the highest-level directory in Linux, and it's always represented with a forward slash (/). All subdirectories branch off the root directory. Subdirectories can continue branching out to as many levels as necessary.

Standard FHS directories

Directly below the root directory, you'll find standard FHS directories. In the diagram, `home`, `bin`, and `etc` are standard FHS directories. Here are a few examples of what standard directories contain:

- `/home`: Each user in the system gets their own home directory.
- `/bin`: This directory stands for “binary” and contains binary files and other executables. Executables are files that contain a series of commands a computer needs to follow to run programs and perform other functions.
- `/etc`: This directory stores the system's configuration files.
- `/tmp`: This directory stores many temporary files. The `/tmp` directory is commonly used by attackers because anyone in the system can modify data in these files.
- `/mnt`: This directory stands for “mount” and stores media, such as USB drives and hard drives.

Pro Tip: You can use the `man hier` command to learn more about the FHS and its standard directories.

User-specific subdirectories

Under `home` are subdirectories for specific users. In the diagram, these users are `analyst` and `analyst2`. Each user has their own personal subdirectories, such as `projects`, `logs`, or `reports`.

Note: When the path leads to a subdirectory below the user's home directory, the user's home directory can be represented as the tilde (`~`). For example, `/home/analyst/logs` can also be represented as `~/logs`.

You can navigate to specific subdirectories using their absolute or relative file paths. The **absolute file path** is the full file path, which starts from the root. For example, `/home/analyst/projects` is an absolute file path. The **relative file path** is the file path that starts from a user's current directory.

Note: Relative file paths can use a dot (.) to represent the current directory, or two dots (..) to represent the parent of the current directory. An example of a relative file path could be ../projects.

Key commands for navigating the file system

The following Linux commands can be used to navigate the file system: `pwd`, `ls`, and `cd`.

`pwd`

The `pwd` command prints the working directory to the screen. Or in other words, it returns the directory that you're currently in.

The output gives you the absolute path to this directory. For example, if you're in your `home` directory and your username is `analyst`, entering `pwd` returns `/home/analyst`.

Pro Tip: To learn what your username is, use the `whoami` command. The `whoami` command returns the username of the current user. For example, if your username is `analyst`, entering `whoami` returns `analyst`.

`ls`

The `ls` command displays the names of the files and directories in the current working directory. For example, in the video, `ls` returned directories such as `logs`, and a file called `updates.txt`.

Note: If you want to return the contents of a directory that's not your current working directory, you can add an argument after `ls` with the absolute or relative file path to the desired directory. For example, if you're in the `/home/analyst` directory but want to list the contents of its `projects` subdirectory, you can enter `ls /home/analyst/projects` or just `ls projects`.

`cd`

The `cd` command navigates between directories. When you need to change directories, you should use this command.

To navigate to a subdirectory of the current directory, you can add an argument after `cd` with the subdirectory name. For example, if you're in the `/home/analyst` directory and want to navigate to its `projects` subdirectory, you can enter `cd projects`.

You can also navigate to any specific directory by entering the absolute file path. For example, if you're in `/home/analyst/projects`, entering `cd /home/analyst/logs` changes your current directory to `/home/analyst/logs`.

Pro Tip: You can use the relative file path and enter `cd ..` to go up one level in the file structure. For example, if the current directory is `/home/analyst/projects`, entering `cd ..` would change your working directory to `/home/analyst`.

Common commands for reading file content

The following Linux commands are useful for reading file content: `cat`, `head`, `tail`, and `less`.

cat

The `cat` command displays the content of a file. For example, entering `cat updates.txt` returns everything in the `updates.txt` file.

head

The `head` command displays just the beginning of a file, by default 10 lines. The `head` command can be useful when you want to know the basic contents of a file but don't need the full contents. Entering `head updates.txt` returns only the first 10 lines of the `updates.txt` file.

Pro Tip: If you want to change the number of lines returned by `head`, you can specify the number of lines by including `-n`. For example, if you only want to display the first five lines of the `updates.txt` file, enter `head -n 5 updates.txt`.

tail

The `tail` command does the opposite of `head`. This command can be used to display just the end of a file, by default 10 lines. Entering `tail updates.txt` returns only the last 10 lines of the `updates.txt` file.

Pro Tip: You can use `tail` to read the most recent information in a log file.

less

The `less` command returns the content of a file one page at a time. For example, entering `less updates.txt` changes the terminal window to display the contents of `updates.txt` one page at a time. This allows you to easily move forward and backward through the content.

Once you've accessed your content with the `less` command, you can use several keyboard controls to move through the file:

- `Space bar`: Move forward one page
- `b`: Move back one page
- `Down arrow`: Move forward one line
- `Up arrow`: Move back one line
- `q`: Quit and return to the previous terminal window

Key takeaways

It's important for security analysts to be able to navigate Linux and the file system of the FHS. Some key commands for navigating the file system include `pwd`, `ls`, and `cd`. Reading file content is also an important skill in the security profession. This can be done with commands such as `cat`, `head`, `tail`, and `less`.

Filter content in Linux

In this reading, you'll continue exploring Linux commands, which can help you filter for the information you need. You'll learn a new Linux command, **find**, which can help you search files and directories for specific information.

Filtering for information

You previously explored how filtering for information is an important skill for security analysts. **Filtering** is selecting data that match a certain condition. For example, if you had a virus in your system that only affected the **.txt** files, you could use filtering to find these files quickly. Filtering allows you to search based on specific criteria, such as file extension or a string of text.

grep

The **grep** command searches a specified file and returns all lines in the file containing a specified string or text. The **grep** command commonly takes two arguments: a specific string to search for and a specific file to search through.

For example, entering **grep OS updates.txt** returns all lines containing **OS** in the **updates.txt** file. In this example, **OS** is the specific string to search for, and **updates.txt** is the specific file to search through.

Let's look at another example: **grep error time_logs.txt**. Here **grep** is used to search for the text pattern. **error** is the term you are looking for in the **time_logs.txt** file. When you run this command, **grep** will scan the **time_logs.txt** file and print only the lines containing the word **error**.

Piping

The pipe command is accessed using the pipe character (**|**). **Piping** sends the standard output of one command as standard input to another command for further processing. As a reminder, **standard output** is information returned by the OS through the shell, and **standard input** is information received by the OS via the command line.

The pipe character (`|`) is located in various places on a keyboard. On many keyboards, it's located on the same key as the backslash character (`\`). On some keyboards, the `|` can look different and have a small space through the middle of the line. If you can't find the `|`, search online for its location on your particular keyboard.

When used with `grep`, the pipe can help you find directories and files containing a specific word in their names. For example, `ls /home/analyst/reports | grep users` returns the file and directory names in the `reports` directory that contain `users`. Before the pipe, `ls` indicates to list the names of the files and directories in `reports`. Then, it sends this output to the command after the pipe. In this case, `grep users` returns all of the file or directory names containing `users` from the input it received.

Note: Piping is a general form of redirection in Linux and can be used for multiple tasks other than filtering. You can think of piping as a general tool that you can use whenever you want the output of one command to become the input of another command.

find

The `find` command searches for directories and files that meet specified criteria. There's a wide range of criteria that can be specified with `find`. For example, you can search for files and directories that

- Contain a specific string in the name,
- Are a certain file size, or
- Were last modified within a certain time frame.

When using `find`, the first argument after `find` indicates where to start searching. For example, entering `find /home/analyst/projects` searches for everything starting at the `projects` directory.

After this first argument, you need to indicate your criteria for the search. If you don't include a specific search criteria with your second argument, your search will likely return a lot of directories and files.

Specifying criteria involves options. **Options** modify the behavior of a command and commonly begin with a hyphen (`-`).

-name and -iname

One key criteria analysts might use with `find` is to find file or directory names that contain a specific string. The specific string you're searching for must be entered in quotes after the `-name` or `-iname` options. The difference between these two options is that `-name` is case-sensitive, and `-iname` is not.

For example, you might want to find all files in the `projects` directory that contain the word "log" in the file name. To do this, you'd enter `find /home/analyst/projects -name "*log*"`. You could also enter `find /home/analyst/projects -iname "*log*"`.

In these examples, the output would be all files in the `projects` directory that contain `log` surrounded by zero or more characters. The `"*log*"` portion of the command is the search criteria that indicates to search for the string "log". When `-name` is the option, files with names that include `Log` or `LOG`, for example, wouldn't be returned because this option is case-sensitive. However, they would be returned when `-iname` is the option.

Note: An asterisk (*) is used as a wildcard to represent zero or more unknown characters.

-mtime

Security analysts might also use `find` to find files or directories last modified within a certain time frame. The `-mtime` option can be used for this search. For example, entering `find /home/analyst/projects -mtime -3` returns all files and directories in the `projects` directory that have been modified within the past three days.

The `-mtime` option search is based on days, so entering `-mtime +1` indicates all files or directories last modified more than one day ago, and entering `-mtime -1` indicates all files or directories last modified less than one day ago.

Note: The option `-mmin` can be used instead of `-mtime` if you want to base the search on minutes rather than days.

Key takeaways

Filtering for information using Linux commands is an important skill for security analysts so that they can customize data to fit their needs. Three key Linux commands for this are `grep`,

piping (`|`), and `find`. These commands can be used to navigate and filter for information in the file system.

- **Consider the privacy and security implications of using AI.** Consider how using AI tools may affect the security of other people or organizations.

Manage directories and files

Previously, you explored how to manage the file system using Linux commands. The following commands were introduced: `mkdir`, `rmdir`, `touch`, `rm`, `mv`, and `cp`. In this reading, you'll review these commands, the nano text editor, and learn another way to write to files.

Creating and modifying directories

`mkdir`

The `mkdir` command creates a new directory. Like all of the commands presented in this reading, you can either provide the new directory as the absolute file path, which starts from the root, or as a relative file path, which starts from your current directory.

For example, if you want to create a new directory called `network` in your `/home/analyst/logs` directory, you can enter `mkdir /home/analyst/logs/network` to create this new directory. If you're already in the `/home/analyst/logs` directory, you can also create this new directory by entering `mkdir network`.

Pro Tip: You can use the `ls` command to confirm the new directory was added.

`rmdir`

The `rmdir` command removes, or deletes, a directory. For example, entering `rmdir /home/analyst/logs/network` would remove this empty directory from the file system.

Note: The `rmdir` command cannot delete directories with files or subdirectories inside. For example, entering `rmdir /home/analyst` returns an error message.

Creating and modifying files

touch and rm

The `touch` command creates a new file. This file won't have any content inside. If your current directory is `/home/analyst/reports`, entering `touch permissions.txt` creates a new file in the `reports` subdirectory called `permissions.txt`.

The `rm` command removes, or deletes, a file. This command should be used carefully because it's not easy to recover files deleted with `rm`. To remove the permissions file you just created, enter `rm permissions.txt`.

Pro Tip: You can verify that `permissions.txt` was successfully created or removed by entering `ls`.

mv and cp

You can also use `mv` and `cp` when working with files. The `mv` command moves a file or directory to a new location, and the `cp` command copies a file or directory into a new location. The first argument after `mv` or `cp` is the file or directory you want to move or copy, and the second argument is the location you want to move or copy it to.

To move `permissions.txt` into the `logs` subdirectory, enter `mv permissions.txt /home/analyst/logs`. Moving a file removes the file from its original location. However, copying a file doesn't remove it from its original location. To copy `permissions.txt` into the `logs` subdirectory while also keeping it in its original location, enter `cp permissions.txt /home/analyst/logs`.

Note: The `mv` command can also be used to rename files. To rename a file, pass the new name in as the second argument instead of the new location. For example, entering `mv permissions.txt perm.txt` renames the `permissions.txt` file to `perm.txt`.

nano text editor

nano is a command-line file editor that is available by default in many Linux distributions. Many beginners find it easy to use, and it's widely used in the security profession. You can perform multiple basic tasks in nano, such as creating new files and modifying file contents.

To open an existing file in nano from the directory that contains it, enter `nano` followed by the file name. For example, entering `nano permissions.txt` from the `/home/analyst/reports` directory opens a new nano editing window with the `permissions.txt` file open for editing. You can also provide the absolute file path to the file if you're not in the directory that contains it.

You can also create a new file in nano by entering `nano` followed by a new file name. For example, entering `nano authorized_users.txt` from the `/home/analyst/reports` directory creates the `authorized_users.txt` file within that directory and opens it in a new nano editing window.

Since there isn't an auto-saving feature in nano, it's important to save your work before exiting. To save a file in nano, use the keyboard shortcut `Ctrl + O`. You'll be prompted to confirm the file name before saving. To exit out of nano, use the keyboard shortcut `Ctrl + X`.

Note: Vim and Emacs are also popular command-line text editors.

Standard output redirection

There's an additional way you can write to files. Previously, you learned about standard input and standard output. **Standard input** is information received by the OS via the command line, and **standard output** is information returned by the OS through the shell.

You've also learned about piping. **Piping** sends the standard output of one command as standard input to another command for further processing. It uses the pipe character (`|`).

In addition to the pipe (`|`), you can also use the right angle bracket (`>`) and double right angle bracket (`>>`) operators to redirect standard output.

When used with `echo`, the `>` and `>>` operators can be used to send the output of `echo` to a specified file rather than the screen. The difference between the two is that `>` overwrites your existing file, and `>>` adds your content to the end of the existing file instead of overwriting it. The `>` operator should be used carefully, because it's not easy to recover overwritten files.

When you're inside the directory containing the `permissions.txt` file, entering `echo "last updated date" >> permissions.txt` adds the string "last updated date" to

the file contents. Entering `echo "time" > permissions.txt` after this command overwrites the entire file contents of `permissions.txt` with the string “time”.

Note: Both the `>` and `>>` operators will create a new file if one doesn’t already exist with your specified name.

Key takeaways

Knowing how to manage the file system in Linux is an important skill for security analysts. Useful commands for this include: `mkdir`, `rmdir`, `touch`, `rm`, `mv`, and `cp`. When security analysts need to write to files, they can use the nano text editor, or the `>` and `>>` operators.

Permission commands

Previously, you explored file permissions and the commands that you can use to display and change them. In this reading, you’ll review these concepts and also focus on an example of how these commands work together when putting the principle of least privilege into practice.

Reading permissions

In Linux, permissions are represented with a 10-character string. Permissions include:

- **read:** for files, this is the ability to read the file contents; for directories, this is the ability to read all contents in the directory including both files and subdirectories
- **write:** for files, this is the ability to make modifications on the file contents; for directories, this is the ability to create new files in the directory
- **execute:** for files, this is the ability to execute the file if it’s a program; for directories, this is the ability to enter the directory and access its files

These permissions are given to these types of owners:

- **user:** the owner of the file

- **group:** a larger group that the owner is a part of
- **other:** all other users on the system

Each character in the 10-character string conveys different information about these permissions. The following table describes the purpose of each character:

Character	Example	Meaning
1st	d rwxrwxrwx	file type • d for directory • - for a regular file
2nd	d r wxrwxrwx	read permissions for the user • r if the user has read permissions • - if the user lacks read permissions
3rd	d r w w xrwxrwx	write permissions for the user • w if the user has write permissions • - if the user lacks write permissions
4th	d r w x rwxrwx	execute permissions for the user • x if the user has execute permissions • - if the user lacks execute permissions
5th	d r wx r wxrwx	read permissions for the group • r if the group has read permissions

Character	Example	Meaning
		•  if the group lacks read permissions
6th	drwxrwxrwx	write permissions for the group •  if the group has write permissions •  if the group lacks write permissions
7th	drwxrwxrwx	execute permissions for the group •  if the group has execute permissions •  if the group lacks execute permissions
8th	drwxrwxrwx	read permissions for other •  if the other owner type has read permissions •  if the other owner type lacks read permissions
9th	drwxrwxrwx	write permissions for other •  if the other owner type has write permissions •  if the other owner type lacks write permissions
10th	drwxrwxrwx	execute permissions for other •  if the other owner type has execute permissions •  if the other owner type lacks execute permissions

Exploring existing permissions

You can use the `ls` command to investigate who has permissions on files and directories. Previously, you learned that `ls` displays the names of files in directories in the current working directory.

There are additional options you can add to the `ls` command to make your command more specific. Some of these options provide details about permissions. Here are a few important `ls` options for security analysts:

- `ls -a`: Displays hidden files. Hidden files start with a period (.) at the beginning.
- `ls -l`: Displays permissions to files and directories. Also displays other additional information, including owner name, group, file size, and the time of last modification.
- `ls -la`: Displays permissions to files and directories, including hidden files. This is a combination of the other two options.

Changing permissions

The **principle of least privilege** is the concept of granting only the minimal access and authorization required to complete a task or function. In other words, users should not have privileges that are beyond what is necessary. Not following the principle of least privilege can create security risks.

The `chmod` command can help you manage this authorization. The `chmod` command changes permissions on files and directories.

Using chmod

The `chmod` command requires two arguments. The first argument indicates how to change permissions, and the second argument indicates the file or directory that you want to change permissions for. For example, the following command would add all permissions to `login_sessions.txt`:

```
chmod u+rx,g+rx,o+rx login_sessions.txt
```

If you wanted to take all the permissions away, you could use

```
chmod u-rwx,g-rwx,o-rwx login_sessions.txt
```

Another way to assign these permissions is to use the equals sign (=) in this first argument. Using = with **chmod** sets, or assigns, the permissions exactly as specified. For example, the following command would set read permissions for **login_sessions.txt** for user, group, and other:

```
chmod u=r,g=r,o=r login_sessions.txt
```

This command overwrites existing permissions. For instance, if the user previously had write permissions, these write permissions are removed after you specify only read permissions with =.

The following table reviews how each character is used within the first argument of **chmod**:

Character	Description
u	indicates changes will be made to user permissions
g	indicates changes will be made to group permissions
o	indicates changes will be made to other permissions
+	adds permissions to the user, group, or other
-	removes permissions from the user, group, or other
=	assigns permissions for the user, group, or other

Note: When there are permission changes to more than one owner type, commas are needed to separate changes for each owner type. You should not add spaces after those commas.

The principle of least privilege in action

As a security analyst, you may encounter a situation like this one: There's a file called **bonuses.txt** within a compensation directory. The owner of this file is a member of the

Human Resources department with a username of `hrrep1`. It has been decided that `hrrep1` needs access to this file. But, since this file contains confidential information, no one else in the `hr` group needs access.

You run `ls -l` to check the permissions of files in the compensation directory and discover that the permissions for `bonuses.txt` are `-rw-rw----`. The group owner type has read and write permissions that do not align with the principle of least privilege.

To remedy the situation, you input `chmod g-rw bonuses.txt`. Now, only the user who needs to access this file to carry out their job responsibilities can access this file.

Key takeaways

Managing directory and file permissions may be a part of your work as a security analyst. Using `ls` with the `-l` and `-la` options allows you to investigate directory and file permissions. Using `chmod` allows you to change user permissions and ensure they are aligned with the principle of least privilege.

Responsible use of sudo

Previously, you explored authorization, authentication, and Linux commands with `sudo`, `useradd`, and `userdel`. The `sudo` command is important for security analysts because it allows users to have elevated permissions without risking the system by running commands as the root user. You'll continue exploring authorization, authentication, and Linux commands in this reading and learn two more commands that can be used with `sudo`: `usermod` and `chown`.

Responsible use of sudo

To manage authorization and authentication, you need to be a **root user**, or a user with elevated privileges to modify the system. The root user can also be called the "super user." You become a root user by logging in as the root user. However, running commands as the root user is not recommended in Linux because it can create security risks if malicious actors compromise that account. It's also easy to make irreversible mistakes, and the system can't

track who ran a command. For these reasons, rather than logging in as the root user, it's recommended you use `sudo` in Linux when you need elevated privileges.

The `sudo` command temporarily grants elevated permissions to specific users. The name of this command comes from “super user do.” Users must be given access in a configuration file to use `sudo`. This file is called the “sudoers file.” Although using `sudo` is preferable to logging in as the root user, it's important to be aware that users with the elevated permissions to use `sudo` might be more at risk in the event of an attack.

You can compare this to a hotel with a master key. The master key can be used to access any room in the hotel. There are some workers at the hotel who need this key to perform their work. For example, to clean all the rooms, the janitor would scan their ID badge and then use this master key. However, if someone outside the hotel's network gained access to the janitor's ID badge and master key, they could access any room in the hotel. In this example, the janitor with the master key represents a user using `sudo` for elevated privileges. Because of the dangers of `sudo`, only users who really need to use it should have these permissions.

Additionally, even if you need access to `sudo`, you should be careful about using it with only the commands you need and nothing more. Running commands with `sudo` allows users to bypass the typical security controls that are in place to prevent elevated access to an attacker.

Note: Be aware of `sudo` if copying commands from an online source. It's important you don't use `sudo` accidentally.

Authentication and authorization with sudo

You can use `sudo` with many authentication and authorization management tasks. As a reminder, **authentication** is the process of verifying who someone is, and **authorization** is the concept of granting access to specific resources in a system. Some of the key commands used for these tasks include the following:

`useradd`

The `useradd` command adds a user to the system. To add a user with the username of `fgarcia` with `sudo`, enter `sudo useradd fgarcia`. There are additional options you can use with `useradd`:

- **-g**: Sets the user's default group, also called their primary group
- **-G**: Adds the user to additional groups, also called supplemental or secondary groups

To use the **-g** option, the primary group must be specified after **-g**. For example, entering **sudo useradd -g security fgarcia** adds **fgarcia** as a new user and assigns their primary group to be **security**.

To use the **-G** option, the supplemental group must be passed into the command after **-G**. You can add more than one supplemental group at a time with the **-G** option. Entering **sudo useradd -G finance,admin fgarcia** adds **fgarcia** as a new user and adds them to the existing **finance** and **admin** groups.

usermod

The **usermod** command modifies existing user accounts. The same **-g** and **-G** options from the **useradd** command can be used with **usermod** if a user already exists.

To change the primary group of an existing user, you need the **-g** option. For example, entering **sudo usermod -g executive fgarcia** would change **fgarcia**'s primary group to the **executive** group.

To add a supplemental group for an existing user, you need the **-G** option. You also need a **-a** option, which appends the user to an existing group and is only used with the **-G** option. For example, entering **sudo usermod -a -G marketing fgarcia** would add the existing **fgarcia** user to the supplemental **marketing** group.

Note: When changing the supplemental group of an existing user, if you don't include the **-a** option, **-G** will replace any existing supplemental groups with the groups specified after **usermod**. Using **-a** with **-G** ensures that the new groups are added but existing groups are not replaced.

There are other options you can use with **usermod** to specify how you want to modify the user, including:

- **-d**: Changes the user's home directory.
- **-l**: Changes the user's login name.

- **-L**: Locks the account so the user can't log in.

The option always goes after the **usermod** command. For example, to change **fgarcia**'s home directory to **/home/garcia_f**, enter **sudo usermod -d /home/garcia_f fgarcia**. The option **-d** directly follows the command **usermod** before the other two needed arguments.

userdel

The **userdel** command deletes a user from the system. For example, entering **sudo userdel fgarcia** deletes **fgarcia** as a user. Be careful before you delete a user using this command.

The **userdel** command doesn't delete the files in the user's home directory unless you use the **-r** option. Entering **sudo userdel -r fgarcia** would delete **fgarcia** as a user and delete all files in their home directory. Before deleting any user files, you should ensure you have backups in case you need them later.

Note: Instead of deleting the user, you could consider deactivating their account with **usermod -L**. This prevents the user from logging in while still giving you access to their account and associated permissions. For example, if a user left an organization, this option would allow you to identify which files they have ownership over, so you could move this ownership to other users.

chown

The **chown** command changes ownership of a file or directory. You can use **chown** to change user or group ownership. To change the user owner of the **access.txt** file to **fgarcia**, enter **sudo chown fgarcia access.txt**. To change the group owner of **access.txt** to **security**, enter **sudo chown :security access.txt**. You must enter a colon (:) before **security** to designate it as a group name.

Similar to **useradd**, **usermod**, and **userdel**, there are additional options that can be used with **chown**.

Key takeaways

Authentication is the process of a user verifying their identity, and authorization is the process of determining what they have access to. You can use the `sudo` command to temporarily run commands with elevated privileges to complete authentication and authorization management tasks. Specifically, `useradd`, `userdel`, `usermod`, and `chown` can be used to manage users and file ownership.

Integrated Linux support

Linux also has several commands that you can use for support.

`man`

The `man` command displays information on other commands and how they work. It's short for "manual." To search for information on a command, enter the command after `man`. For example, entering `man chown` returns detailed information about `chown`, including the various options you can use with it. The output of the `man` command is also called a "man page."

`apropos`

The `apropos` command searches the man page descriptions for a specified string. Man pages can be lengthy and difficult to search through if you're looking for a specific keyword. To use `apropos`, enter the keyword after `apropos`.

You can also include the `-a` option to search for multiple words. For example, entering `apropos -a graph editor` outputs man pages that contain both the words "graph" and "editor" in their descriptions.

`whatis`

The `whatis` command displays a description of a command on a single line. For example, entering `whatis nano` outputs the description of `nano`. This command is useful when you don't need a detailed description, just a general idea of the command. This might be as a

reminder. Or, it might be after you discover a new command through a colleague or online resource and want to know more.

Key takeaways

There are many resources available for troubleshooting issues or getting support for Linux. Linux has a large global community of users who ask and answer questions on online resources, such as the Unix and Linux Stack Exchange. You can also use integrated support commands in Linux, such as `man`, `apropos`, and `whatis`.